

polars-bio w trybie rozproszonym: Ballista i Sail

Wy tłumaczenie krok po kroku — od podstaw

Miłosz Kowalewski

26 sierpnia 2026

Spis treści

Wstęp — o co w ogóle chodzi	2
Rozdział 1 — Cegielki, na których to wszystko stoi	2
Apache Arrow — wspólny język danych	2
DataFusion — silnik, który wykonuje zapytania	2
polars-bio i COITrees — sedno algorytmu	3
Rozdział 2 — Ballista: rozproszenie „z tej samej rodziny”	3
Problem: jak przesłać niestandardowy „przepis”?	3
Rozwiązanie: własny „tłumacz” (codec)	3
Dogonienie: COITrees też w wersji rozproszonej	4
Rozdział 3 — Sail: inna rodzina, inna droga	4
Jak to działa	5
Problemy napotkane po drodze	5
Eksperyment: czy Sail da się uruchomić NAPRAWDĘ rozproszony?	5
Jak to sprawdzono	5
Wniosek	6
Co to jest Kubernetes i dlaczego to komplikuje sprawę	6
Co to jest taki „limit pamięci” (cgroup)	6
Co poszło nie tak	7
Co się stało dalej	7
Co z tego wynika	7
Rozdział 4 — Reszta operacji też rusza z miejsca	7
Najpierw: skąd w ogóle wiadomo, czy coś liczy się rozproszone?	7
Sprytna sztuczka: dane ułożone tak, żeby błąd był widoczny	8
Co trzeba było zrobić	8
Dwa różne sposoby rozpraszania	8
Jedna operacja, której nie da się rozproszyć — i dlaczego to dobra wiadomość	9
Rozdział 5 — Sail: oskarżyliśmy niewinnego	9
Co myśleliśmy	9
Jak sprawdziliśmy	9
Co się naprawdę działo	10
Naprawa	10
Dlaczego to ważne dla pracy	10
Rozdział 6 — Jak przenieść to wszystko do chmury Google	11

Czy trzeba jakiegoś specjalnego programu od Google?	11
Co gdzie postawić	11
Co to jest Kubernetes (raz jeszcze, krótko)	11
Co jest już przygotowane	11
Ile to kosztuje	11

Rozdział 7 — Co to wszystko znaczy dla całej pracy magisterskiej 12

Wstęp — o co w ogóle chodzi

Wyobraź sobie, że masz ogromną książkę telefoniczną — tak dużą, że nie mieści się w pamięci jednego komputera. Chcesz w niej znaleźć wszystkie pary osób, które mieszkają na tej samej ulicy. Jeden komputer by się z tym męczył (albo w ogóle by się nie dało, bo dane nie mieszczą się w RAM-ie). Rozwiązanie: pociąć książkę na kawałki, rozdać je wielu komputerom naraz, każdy przeszuka swój kawałek, a na końcu ktoś skleja wyniki. To właśnie nazywa się **przetwarzaniem rozproszonym**.

W tej pracy magisterskiej „książką telefoniczną” są dane genomiczne — np. miliony odcinków DNA opisanych jako (chromosom, początek, koniec). Operacja, o którą chodzi najczęściej, to **overlap**: „znajdź wszystkie pary odcinków, które się nakładają”. Jest jeszcze kilka innych podobnych operacji (merge — skleja nakładające się odcinki w jeden; nearest — znajdź najbliższy sąsiedni odcinek; coverage — ile razy dany fragment jest pokryty; subtract — odejmij jedne odcinki od drugich).

Do liczenia tych operacji na jednym komputerze służy biblioteka **polars-bio** (współautor: promotor tej pracy). Jest bardzo szybka, ale działa tylko na jednej maszynie. Cel tej pracy: sprawdzić, czy da się **polars-bio** „dowieźć” do dwóch popularnych silników do przetwarzania rozproszonego — **Apache Ballista** i **Sail** — tak żeby te same obliczenia dało się rozłożyć na wiele komputerów, bez przepisywania **polars-bio** od zera i bez „forkowania” (kopiowania i modyfikowania na stałe) tych silników.

Rozdział 1 — Cegielki, na których to wszystko stoi

Apache Arrow — wspólny język danych

Różne programy (Python, Rust, Java) zwykle trzymają dane w pamięci na swój własny sposób, więc żeby jeden program przekazał dane drugiemu, musi je „przetłumaczyć” — co jest wolne. **Apache Arrow** to uzgodniony, wspólny format przechowywania danych tabelarycznych w pamięci, który rozumieją wszystkie zaangażowane tu technologie (**polars-bio**, **DataFusion**, **Ballista**, **Sail**). Dzięki temu nie trzeba niczego tłumaczyć — to jak gdyby wszyscy w firmie nagle zaczęli mówić tym samym językiem, zamiast przez tłumacza.

DataFusion — silnik, który wykonuje zapytania

DataFusion to biblioteka (napisana w języku Rust), która potrafi wziąć zapytanie w stylu SQL („wybierz te wiersze, połącz te dwie tabele, policz sumę”) i wykonać je szybko na danych w formacie Arrow. Nie jest to gotowy program z interfejsem — to raczej „silnik pod maską”, którego używają inne programy. Zarówno **polars-bio**, jak i **Ballista**, jak i **Sail** — wszystkie trzy są zbudowane na **DataFusion** (**Sail** nieco inaczej, o czym niżej). Dzięki temu współdzielą fundament: sposób opisywania planu zapytania (czyli „co dokładnie trzeba policzyć i w jakiej kolejności”).

polars-bio i COITrees — sedno algorytmu

polars-bio to biblioteka w Pythonie do analiz genomicznych, zbudowana na DataFusion. Kiedy wywołasz `pb.overlap(a, b)`, tak naprawdę pod spodem woła ona zewnętrzny komponent — `datafusion-bio-function-ranges` — który implementuje prawdziwy algorytm. Jednym z dostępnych algorytmów jest **COITrees** — sprytna struktura danych (drzewo przedziałów), która pozwala sprawdzić „czy ten odcinek nakłada się z czymś” dużo szybciej niż porównywanie każdego z każdym (to ostatnie dla dużych danych byłoby beznadziejnie wolne — jak sprawdzanie wszystkich par osób w mieście, zamiast najpierw pogrupować je ulicami).

Ważne odkrycie na starcie tej pracy: pierwszy prototyp w ogóle nie używał tego prawdziwego silnika — liczył nakładanie się odcinków „na piechotę” (porównanie każdy z każdym). Jednym z pierwszych kroków było więc podpięcie prawdziwego silnika **polars-bio**, żeby porównanie z silnikami rozproszonymi było uczciwe.

Rozdział 2 — Ballista: rozproszenie „z tej samej rodziny”

Apache Ballista to nakładka na DataFusion, która potrafi rozdzielić zapytanie na wiele komputerów. Ma dwie główne role:

```
KLIENT (Twój program)
| wysła plan zapytania
v
SCHEDULER ---- rozdziela zadania --> EXECUTOR 1
|                                     EXECUTOR 2
|<----- zbiera wyniki ----- EXECUTOR 3
v
wynik z powrotem do klienta
```

Scheduler to „dyrygent” — dostaje zapytanie, dzieli dane na kawałki (np. po chromosomie: „chr1” do jednego workera, „chr2” do drugiego) i rozsyła zadania. **Executory** to „robotnicy” — każdy dostaje swój kawałek danych i swoją część zadania, liczy ją lokalnie, i odsyła wynik z powrotem. Ruch danych między schedulerem a executorami nazywa się **shuffle**.

Ponieważ scheduler i executor to osobne procesy (czasem nawet osobne komputery), trzeba im przesłać nie tylko dane, ale też sam **plan zapytania** — czyli dokładny „przepis”, co i jak policzyć. Zwykle, standardowe operacje (jak „posortuj” czy „policz sumę”) DataFusion potrafi zamienić na taki przesyłalny przepis samodzielnie. Problem pojawia się, gdy chcemy użyć czegoś niestandardowego — jak właśnie nasz operator `overlap` z **polars-bio**.

Problem: jak przesłać niestandardowy „przepis”?

Kiedy spróbowano po raz pierwszy wysłać zapytanie z `overlap` do prawdziwego klastra Ballista, dostano błąd: *„LogicalExtensionCodec is not provided”* — po polsku: „nie wiem, jak zapisać ten kawałek przepisu do przesłania, bo to coś niestandardowego, nie mam do tego instrukcji tłumaczenia”. To jak wysłanie komuś przepisu kulinarnego z jednym krokiem napisanym w nieznanym języku — reszta przepisu jest zrozumiała, ale tego jednego kroku odbiorca nie umie odczytać.

Rozwiązanie: własny „tłumacz” (codec)

Napisano własny **kodek** (`LogicalExtensionCodec`) — czyli dokładnie taką instrukcję tłumaczenia dla tego jednego niestandardowego kroku: „jeśli zobaczysz krok o nazwie X, to zapisz go jako te trzy liczby/napisy, a po drugiej stronie odtwórz go z powrotem z tych trzech liczb/napisów”. Nie

trzeba było przy tym w żaden sposób modyfikować samej Ballisty — to mechanizm rozszerzeń, który Ballista udostępnia z założenia, dokładnie po to, żeby dało się do niej dopisywać nowe rzeczy bez grzebania w jej wnętrzu.

Efekt pierwszej udanej próby: zapytanie z overlap policzone poprawnie na prawdziwym klastrze (scheduler + kilku executorów, prawdziwy shuffle danych między nimi) — ale z jednym kompromisem: executor liczył wynik zwykłym, generycznym sposobem łączenia tabel (HashJoinExec), a nie sprytnym COITrees. Czyli rozproszenie działało, ale bez najszybszego algorytmu w środku.

Dogonienie: COITrees też w wersji rozproszonej

W tej sesji postawiono sobie za cel, żeby to naprawić — użyć COITrees **także** w wersji w pełni rozproszonej, nie tylko lokalnie na jednym komputerze.

Sedno problemu: sam mechanizm liczący przez COITrees (IntervalJoinExec) był dostępny „z zewnątrz” prawie w całości — poza jednym drobnym elementem: strukturą opisującą, które kolumny to początek/koniec odcinka (ColIntervals), która była ukryta „za ścianą” — schowana w module, do którego kod spoza tej biblioteki nie ma dostępu (to trochę jak szuflada w czymś biurku, zamknięta na klucz, mimo że rzecz w środku wcale nie jest tajna — po prostu nikt nie zostawił do niej klucza dla osób z zewnątrz).

Rozwiązanie: zamiast rezygnować, zrobiono lokalną kopię tej jednej, pomocniczej biblioteki (**nie** Ballisty ani Saila — tylko małego komponentu, który dostarcza sam algorytm) i zmieniono w niej dosłownie jedno słowo w kodzie źródłowym: `mod` na `pub mod` (czyli „ten moduł jest prywatny” → „ten moduł jest publiczny”) — odpowiednik zostawienia klucza do tej szuflady również dla gości. To udokumentowana, przejrzysta, minimalna zmiana — nie fork silnika rozproszonego, tylko drobna poprawka widoczności w bibliotece pomocniczej, gotowa do zaproponowania z powrotem autorom oryginału.

Po tej poprawce udało się napisać drugi, bardziej zaawansowany kodek — tym razem dla **fizycznego** planu (PhysicalExtensionCodec), czyli przepisu na poziomie „executor do executora”, a nie tylko „klient do schedulera”. Po drodze napotkano jeszcze jeden błąd uruchomieniowy (PartitionMode Auto — coś w rodzaju „nie zdążono ustalić, w jaki sposób dane mają zostać podzielone między executory, zanim faktycznie zaczęto liczyć” — naprawione przez jawne wymuszenie konkretnego trybu podziału).

Wynik końcowy Części 1: to samo zapytanie overlap, na prawdziwym, rozproszonym klastrze Ballista, tym razem faktycznie liczone przez COITrees na każdym executorze — potwierdzone identycznym wynikiem jak przy liczeniu na jednym komputerze. To jest największe osiągnięcie tej sesji: pełna dystrybucja **bez** utraty szybkiego algorytmu.

Rozdział 3 — Sail: inna rodzina, inna droga

Sail to inny silnik rozproszony — zamiast być bezpośrednio zbudowany na tych samych mechanizmach co Ballista, mówi tzw. protokołem **Spark Connect** — czyli protokołem, którym komunikuje się (Py)Spark, bardzo popularne narzędzie do przetwarzania rozproszonego. Dzięki temu programy pisane pod PySparka mogą łączyć się z Sailem, nie wiedząc nawet, że to nie jest prawdziwy Spark.

Ponieważ Sail nie należy do „rodziny DataFusion” w takim samym sensie jak Ballista, nie da się mu bezpośrednio wstrzyknąć niestandardowego kroku planu zapytania tak jak zrobiono to z kodekami w Ballistrze (mechanizm do tego, nazwany roboczo `SailExtension`, jest dopiero projektowany przez zespół Saila — nie jest gotowy). Zamiast tego użyto innego, w pełni legalnego mechanizmu: **UDTF** (*User-Defined Table Function* — funkcja zdefiniowana przez

użytkownika, która na wejściu i wyjściu operuje na całych tabelach, nie pojedynczych wartościach).

Jak to działa

Sail dzieli dane na grupy (po chromosomie) -- standardowa operacja, jak "group by"

|

v

Dla każdej grupy: Sail woła funkcję UDTF, którą sami zarejestrowaliśmy

|

v

Wewnątrz UDTF: nasz kod woła z powrotem pb.overlap() -- prawdziwe, niezmiennicze polars-bio

|

v

Wynik wraca do Saila, Sail składa wszystko w jedną tabelę

Innymi słowy: Sail robi to, co umie najlepiej (dzielenie i rozsyłanie pracy), a właściwe liczenie overlap dalej robi polars-bio — tylko wywoływane osobno na każdej porcji danych.

Problemy napotkane po drodze

1. **UDTF nie przyjmuje całych tabel jako argumentu** — w wersji Saila użytej w tej pracy dało się przekazać do UDTF tylko pojedyncze wartości (np. nazwę chromosomu, listę odcinków jako jedną skomplikowaną wartość), a nie „prawdziwą” tabelę z wieloma wierszami jako osobny argument. Trzeba było to obejść, pakując dane w listy przekazywane jako pojedyncza wartość.
2. **Znikające wyniki przy wielu kawałkach naraz** — kiedy dane były podzielone na więcej niż jeden kawałek i UDTF uruchamiał się na kilku kawałkach jednocześnie, część wyników znikała. Obejściem było wymuszenie jednego kawałka danych przed wywołaniem UDTF: naprawiało poprawność, ale kosztem tego, że nie było widać żadnego przyspieszenia z posiadania wielu komputerów.

Przez długi czas myśleliśmy, że to błąd Saila. Okazało się, że nie — i to jest jedna z ciekawszych rzeczy, które wyszły w tej pracy. Wyjaśnienie w Rozdziale 5.

3. **Rejestracja funkcji „w złym momencie”** — sposób, w jaki Sail sprawdza czy program działa w trybie lokalnym czy rozproszonym, robi to zbyt wcześnie (w momencie pisania kodu funkcji, a nie w momencie jej faktycznego zarejestrowania) — trzeba było zmienić sposób organizacji kodu, żeby to zadziałało poprawnie.

Wynik: ta sama operacja overlap, wywołana przez Saila, dająca identyczny wynik jak lokalne polars-bio.

Eksperyment: czy Sail da się uruchomić NAPRAWDĘ rozproszony?

Po zakończeniu podstawowej weryfikacji zadano ważniejsze pytanie: czy Sail w trybie „local-cluster” (który brzmi jak „klaster”) rzeczywiście uruchamia osobne procesy komputerowe, czy tylko to udaje?

Jak to sprawdzono

Zrobiono „zdjęcie” działającego systemu w trakcie liczenia — listę wszystkich działających procesów oraz listę wszystkich otwartych „gniazd sieciowych” (portów), na których coś nasłuchuje. Wynik: **wszystkie cztery role** — „dyrygent” (driver), dwóch „robotników” (worker 1, worker 2) i serwer komunikacyjny — należały do **jednego i tego samego procesu**

na komputerze (jeden numer PID). To trochę jak firma, która ma cztery szyldy na drzwiach różnych biur, ale w środku wszystkie „biura” to w rzeczywistości jedna i ta sama osoba, przebijająca się w różne kostiumy i biegająca między pokojami.

Dodatkowym, przypadkowym potwierdzeniem było to, że próba użycia „kłódki” (`threading.Lock`) do zablokowania jednoczesnego dostępu dwóch workerów do tych samych danych **nie zadziałała** — dostano błąd, że kłódki nie da się „spakować i wysłać” (zserializować). To dowód, że mimo bycia jednym procesem, workery faktycznie przesyłają między sobą kod jako gotowe, samodzielne paczki (tak jakby naprawdę były osobnymi komputerami) — więc czegoś tak „fizycznego” jak kłódka nie da się między nimi przekazać. Zamiast kłódki użyto innego triku: próbuj ponownie po chwili, jeśli akurat ktoś inny w tym momencie korzysta z tych samych danych.

Wniosek

Prawdziwa, wieloprocusowa/wielokomputerowa dystrybucja w Sailu wymaga innego trybu — **Kubernetes** (`KubernetesCluster`). Bez Kubernetesa, „local-cluster” to tylko symulacja wyglądająca jak klastr, ale fizycznie działająca w jednym procesie.

Co to jest Kubernetes i dlaczego to komplikuje sprawę

Kubernetes to system do zarządzania wieloma „kontenerami” (odizolowanymi, przenośnymi paczkami z programem i wszystkim, czego potrzebuje do działania) rozsianymi po wielu maszynach — sam decyduje, gdzie co uruchomić, restartuje, jeśli coś padnie, itd. To potężne narzędzie, ale też ciężkie — normalnie uruchamia się je na serwerach z dużą ilością pamięci.

Skoro maszyna testowa ma tylko **3,5GB RAM łącznie** (bardzo mało jak na tego typu zadania — dla porównania, przeciętny nowy laptop ma 16GB), a raz już zdarzyło się, że zwykła kompilacja programu zawiesiła cały komputer, postanowiono najpierw sprawdzić na chłodno, ile realnie pamięci taki eksperyment by zjadł, zanim cokolwiek zainstalowano.

Znaleziono lżejszą wersję Kubernetesa — k3s — reklamowaną jako potrzebującą tylko 2GB RAM minimum. To wciąż bardzo blisko granicy całej dostępnej pamięci na tej maszynie, więc zaplanowano dodatkowe zabezpieczenie: **twardy limit pamięci** dla samego procesu k3s, żeby jeśli przekroczy on bezpieczną granicę, został szybko i czysto zatrzymany, zamiast powoli dusić cały system (co jest dokładnie tym, co się stało przy poprzednim zawieszeniu — komputer nie „umarł” od razu, tylko stawał się coraz wolniejszy, aż przestał odpowiadać).

Co to jest taki „limit pamięci” (cgroup)

Wyobraź sobie, że każdemu programowi można przypisać własną, oddzielną „kopertę” z ograniczoną ilością pieniędzy (pamięci) do wydania. Jeśli program spróbuje wydać więcej niż ma w kopercie, zostaje natychmiast zatrzymany — reszta budżetu domowego (systemu) zostaje nietknięta. Mechanizm w Linuksie, który to umożliwia, nazywa się **cgroup** (*control group*). Dodatkowo poproszono system, żeby dla tej konkretnej „koperty” w ogóle nie pozwalał na sięganie po dodatkowy, awaryjny fundusz (**swap**, czyli miejsce na dysku używane jako zapasowa, wolniejsza pamięć) — żeby zamiast powolnego wpadania w długi, program od razu dostał twarde „nie” i się zatrzymał.

Przy okazji zwiększono też ogólny, awaryjny fundusz (swap) dla reszty systemu z 1GB do 3GB, oraz zmieniono ustawienie mówiące systemowi, jak chętnie ma sięgać po ten fundusz zamiast czyścić pamięć na bieżąco (`swappiness`) — z wartości „dość chętnie” na „raczej niechętnie”, żeby ewentualne problemy kończyły się szybko i czysto, a nie długim dogorywaniem systemu.

Co poszło nie tak

Instalacja k3s przebiegła bez problemu. Problem pojawił się przy próbie odpalenia go w typowy sposób (przez `systemd` — standardowy „zarządca usług” w Linuksie, coś w rodzaju recepcjonisty, który uruchamia i pilnuje programów działających w tle). Okazało się, że to konkretne środowisko (WSL — sposób uruchamiania Linuksa wewnątrz Windowsa) **w ogóle nie ma tego recepcjonisty włączonego** — wcześniejsze sprawdzenie tego faktu było błędne (sprawdzono tylko, czy istnieje odpowiednie narzędzie, a nie czy faktycznie działa jako „prawdziwy” zarządca).

Naprawienie tego na stałe wymagałoby pełnego restartu całego środowiska Linux w Windowsie — co ubiłoby też bieżącą sesję pracy, więc uznano to za zbyt ryzykowne. Zamiast tego spróbowano odpalić k3s „z ręki”, bezpośrednio umieszczając go w przygotowanej „kopercie” (cgroup) — ale próba zapisania limitu do tej koperty została odrzucona przez system z komunikatem „odmowa dostępu”, **nawet dla administratora**. Innymi słowy: mechanizm bezpieczeństwa, na którym opierał się cały plan, okazał się niedostępny w tym konkretnym środowisku.

Co się stało dalej

k3s wystartował — ale **bez żadnego limitu**. W ciągu około minuty, jeszcze zanim zdążył w pełni się uruchomić (był w trakcie instalowania swoich domyślnych, dodatkowych komponentów), zjadł niemal całą dostępną pamięć komputera (z 2,1GB dostępnego zapasu do 171MB). To dokładnie ten scenariusz, którego cała ta ostrożność miała nie dopuścić.

Zareagowano natychmiast: zatrzymano proces oficjalnym, wbudowanym w k3s poleceniem do czystego wyłączenia wszystkiego, co uruchomił (włącznie z jego procesami pomocniczymi). Pamięć od razu wróciła do normy, komputer nie ucierpiał. Na koniec odinstalowano k3s całkowicie, zostawiając system czysty.

Co z tego wynika

To jest **wynik negatywny, ale bardzo wartościowy i rzetelnie sprawdzony**: nie „w teorii pewnie by nie działało”, tylko „faktycznie spróbowaliśmy, ostrożnie, z zabezpieczeniami, i realnie zabrakło zasobów zanim mechanizm zabezpieczający zdążył pomóc”. To ważny wniosek porównawczy do pracy: na tej samej maszynie Ballista osiągnęła pełną, prawdziwą dystrybucję (Rozdział 2), a Sail — nie, z jasno wskazanym i udokumentowanym powodem: jego jedyna lokalnie dostępna, naprawdę-wieloprocessowa ścieżka wymaga Kubernetesa, a ten akurat na tym sprzęcie się nie mieści.

Rozdział 4 — Reszta operacji też rusza z miejsca

Do tej pory rozproszona (czyli licząca się naprawdę na wielu „robotnikach”) była tylko jedna operacja: **overlap**. Pozostałe cztery — merge, subtract, nearest, coverage — działały tylko na jednym komputerze. Ten rozdział opisuje, jak i one ruszyły.

Najpierw: skąd w ogóle wiadomo, czy coś liczy się rozproszone?

To jest ważniejsze pytanie, niż się wydaje. Zapytanie może dać **w stu procentach poprawny wynik** i jednocześnie policzyć się w całości na jednym robotniku — bo poprawność i szybkość to dwie różne rzeczy. Gdyby ktoś oceniał tylko wynik, mógłby ogłosić sukces tam, gdzie nic się nie rozproszyło.

Na szczęście Ballista potrafi pokazać „paragon” z wykonania zapytania. Wystarczy poprosić ją o `EXPLAIN ANALYZE`, a wypisze coś takiego:

```
=====Etap 1, liczba kawałków: 2=====
Rozeslij dane, dzielac je wg chromosomu na 4 kubelki
Czytaj pliki: [a_part1.csv], [a_part2.csv]      <- dwa pliki, dwa osobne zadania
```

```
=====Etap 2, liczba kawałków: 4=====
MergeExec (nasza operacja genomiczna)
Odbierz dane z sieci, podzielone wg chromosomu <- dane przyszły od innego etapu
```

To jest twardy dowód: widać, że zapytanie zostało pocięte na etapy, że dane faktycznie poszły przez sieć i że na końcu pracowały cztery kawałki naraz. Napisaliśmy zestaw testów, które czytają taki paragon i **sprawdzają go automatycznie** — żeby nikt (łącznie z nami) nie musiał wierzyć na słowo.

Sprytna sztuczka: dane ułożone tak, żeby błąd był widoczny

Jest jeszcze drugi, jeszcze lepszy dowód — ukryty w samych danych testowych.

Mamy dwa odcinki, które się nakładają: gene_A1 od 100 do 200 i gene_A2 od 150 do 300. Operacja merge powinna je skleić w jeden odcinek od 100 do 300.

Celowo umieściliśmy je w **dwóch różnych plikach wejściowych**. Dzięki temu na starcie trafiają do dwóch różnych robotników. Żeby dały się skleić, muszą najpierw zostać przeniesione przez sieć do jednego miejsca — właśnie po to jest „rozsyłanie wg chromosomu”.

Efekt: jeśli rozpraszanie kiedykolwiek przestanie działać, wynik nie będzie po prostu wolniejszy — będzie **zły**, i to w bardzo widoczny sposób (dwa osobne odcinki zamiast jednego sklejonego). Test poprawności stał się w ten sposób testem rozproszenia.

Co trzeba było zrobić

Przypomnijmy problem z Rozdziału 2: żeby przesłać niestandardowy krok przepisu do robotnika, trzeba napisać „tłumacza” (kodek). Żeby napisać tłumacza dla danej operacji, trzeba móc ją z zewnątrz **nazwać i odtworzyć**.

Wcześniej uznaliśmy, że dla tych czterech operacji to niewykonalne. **Ta ocena była przedwczesna** — powstała, zanim zrobiliśmy sobie lokalną kopię biblioteki. Po ponownym sprawdzeniu okazało się, że wystarczy ta sama drobna zmiana co poprzednio: dopisanie słowa pub („to jest publiczne”) w kilkudziesięciu miejscach. Zero zmian w tym, *co* kod robi — wyłącznie w tym, *kto może się do niego odwołać*. Wracając do wcześniejszej analogii: znów chodziło o zostawienie klucza do szuflady, a nie o zmianę jej zawartości.

Dwa różne sposoby rozpraszania

Operacje podzieliły się na dwie grupy — i to nie jest przypadek, tylko konsekwencja tego, co która operacja robi:

Grupa pierwsza — „rozeslij wszystkim wg chromosomu” (merge, subtract). Te operacje przetwarzają odcinki chromosom po chromosomie. Wystarczy więc rozesłać dane tak, żeby wszystko z chr1 trafiło do jednego robotnika, a wszystko z chr2 do drugiego. Potem każdy pracuje niezależnie. To najczystsza postać rozproszenia.

Przy subtract dzieje się to po **obu** stronach naraz (bo odejmujemy jedne odcinki od drugich), więc plan ma cztery etapy zamiast trzech — najbardziej rozbudowany przypadek w całej pracy.

Grupa druga — „rozdaj wszystkim kopię książki” (nearest, coverage). Tu jest inaczej. Żeby znaleźć najbliższego sąsiada, trzeba móc przeszukać *całą* drugą tabelę, a nie tylko jej kawałek. Rozwiązanie: mniejsza tabela jest **wysyłana w całości do każdego robotnika**

razem z przepisem, a zrównoleglone jest przetwarzanie tej większej. To znany wzorzec i ma swoją nazwę: *broadcast join*.

Ma też swoją granicę, którą trzeba uczciwie odnotować: przepis wraz z dołączoną tabelą wędruje przez sieć jako jedna paczka, a Ballista ma limit **16 MB** na taką paczkę. Dla małych tabel adnotacji to bez znaczenia; dla dużych ta metoda przestanie działać i trzeba by innej.

Jedna operacja, której nie da się rozproszyć — i dlatego to dobra wiadomość

Piąta operacja z biblioteki, **cluster**, nie poddaje się i nie podda się żadną sztuczką z widocznością. Warto zrozumieć dlaczego, bo to mówi coś ogólnego.

`cluster` numeruje znalezione skupiska odcinków: 1, 2, 3... Żeby nadać numery, musi wiedzieć, ile skupisk znaleźli **wszyscy pozostali**, bo inaczej dwóch robotników przydzieliliby ten sam numer dwóm różnym skupiskom. W kodzie jest to zrobione tak, że wszyscy zatrzymują się i czekają na siebie nawzajem w jednym miejscu — jak zbiórka przed wyjściem.

Taka zbiórka działa, dopóki wszyscy są w tym samym pokoju (procesie). Po rozproszeniu na osobne maszyny każdy robotnik miałby **własną, osobną zbiórkę** — i albo czekałby w nieskończoność na kolegów, którzy nigdy nie przyjdą, albo ruszyłby sam i nadał numery kolidujące z cudzymi.

To jest ważny wniosek, wart zapisania w pracy: **granica rozpraszania nie przebiega tam, gdzie kończy się dostępność API, tylko tam, gdzie algorytm zakłada, że wszyscy widzą wspólną pamięć**. Czterech operacji dało się rozproszyć, bo każda przetwarza swój kawałek niezależnie. Piątej nie, bo z założenia wymaga uzgodnienia między wszystkimi.

Rozdział 5 — Sail: oskarżyliśmy niewinnego

To jest historia o pomyłce, którą udało się naprawić — i chyba najciekawszy pojedynczy wynik całej tej części pracy.

Co myśleliśmy

Przypomnijmy problem #2 z Rozdziału 3: gdy UDTF uruchamiał się na kilku kawałkach danych naraz, część wyników zniknęła. Wniosek wydawał się oczywisty: **Sail ma błąd**. Zapisaliśmy to jako usterkę Saila i zastosowaliśmy obejście — wymuszenie jednego kawałka.

Obejście działało, ale miało poważny koszt: skoro wszystko liczy się w jednym kawałku, to Sail nie może pokazać *żadnego* przyspieszenia z posiadania wielu komputerów. Dla pracy, która ma porównywać wydajność dwóch silników, to poważny problem.

Jak sprawdziliśmy

Zamiast dalej zakładać, zrobiliśmy serię eksperymentów — za każdym razem **bez** obejścia, i za każdym razem po kilka powtórzeń (bo błąd pojawiał się losowo, więc jedno przejście niczego by nie dowiodło):

Co uruchomiliśmy	Ile razy poprawnie
UDTF liczący merge własnym kodem w Pythonie , bez <code>polars-bio</code>	3 na 3
UDTF wołający <code>polars-bio</code>	1 na 3
UDTF wołający <code>polars-bio</code> + nasza funkcja „czyszcząca”	0 na 3

Co uruchomiliśmy	Ile razy poprawnie
Zupełnie inny mechanizm Saila (applyInPandas) + polars-bio	0 na 3

Pierwszy wiersz przesądza sprawę. **Dokładnie ten sam kształt zapytania**, ta sama liczba kawałków, ten sam mechanizm Saila — ale operacja policzona zwykłym kodem w Pythonie zamiast przez polars-bio. Wynik: bezbłędnie, za każdym razem.

Czyli Sail rozsyła dane i zbiera wyniki prawidłowo. Winowajca jest gdzie indziej.

Co się naprawdę działo

polars-bio trzyma **jedną wspólną „tablicę roboczą”** dla całego programu. Gdy wywołujesz `pb.merge()`, biblioteka zapisuje na niej dane wejściowe pod ustalonymi nazwami, liczy i sprząta.

Dopóki liczy jedna rzecz naraz, wszystko gra. Ale gdy dwóch robotników w tym samym procesie zacznie liczyć **jednocześnie**, obaj piszą po tej samej tablicy. Jeden zamazuje dane drugiego — i ten drugi dostaje pusty albo błędny wynik. Nie ma żadnego komunikatu o błędzie; wynik po prostu cicho znika.

Gorzej: nasza własna funkcja „czyszcząca tablicę przed użyciem”, którą dodaliśmy wcześniej w dobrej wierze, **pogarszała sprawę**. Skoro jawnie wycierała tablicę, to robotnik potrafił wytrzeć dane koledze w trakcie liczenia — i z losowego błędu robił się błąd systematyczny. Stąd 0 na 3 zamiast 1 na 3.

Naprawa

Rozwiązanie okazało się proste: **kolejka do tablicy**. Zanim robotnik zacznie liczyć, bierze „klucz”; kto nie ma klucza, czeka. Po skończeniu oddaje klucz następnemu. W programowaniu nazywa się to *lock*.

Był jeden haczyk. Sail wysyła funkcję użytkownika do robotników, „pakując” ją — a kluczy tego typu nie da się zapakować (dostawaliśmy błąd wprost o tym mówiący). Rozwiązanie: klucz nie podróżuje razem z funkcją. Leży w osobnym, wspólnym pliku (module), a funkcja tylko mówi „weź klucz stamtąd”. Wtedy pakowana jest sama *notatka*, *gdzie leży klucz*, a nie klucz.

Wynik: **5 uruchomień na 5 poprawnych, bez obejścia**. Sail znów liczy na wielu kawałkach naraz.

Dlaczego to ważne dla pracy

Po pierwsze, z **Saila zdjęty został niesłuszny zarzut**. Praca ma porównywać silniki uczciwie, a przypisanie komuś błędu, którego nie popełnił, jest po prostu nierzetelne.

Po drugie, **Sail odzyskał równoległość** — bez tego przyszłe pomiary wydajności nie miałyby dla niego sensu.

Po trzecie — i to być może najciekawsze — po drodze zidentyfikowaliśmy **prawdziwe ograniczenie polars-bio**: biblioteka nie jest przygotowana na to, że kilka rzeczy będzie ją wołać naraz w jednym procesie. Da się to obejść z zewnątrz (i obeszlśmy), ale docelowo warto naprawić u źródła. Jest to o tyle istotne, że promotor tej pracy jest współautorem polars-bio.

Rozdział 6 — Jak przenieść to wszystko do chmury Google

Czy trzeba jakiegoś specjalnego programu od Google?

Nie. Można zostać przy VS Code. Są cztery drogi:

1. **VS Code jak dotąd + jedno narzędzie do wpisywania komend** (gcloud). Pisziesz kod dokładnie tak jak teraz, a wysyłasz go do chmury komendą. To wystarczy do wszystkiego.
2. **Dodatek „Cloud Code”** — oficjalna, darmowa wtyczka Google do VS Code. Wciąga podgląd chmury do okna edytora. Przydatna, ale nieobowiązkowa. To dalej jest VS Code.
3. **Podłączenie VS Code do komputera stojącego w chmurze** (Remote-SSH) — **to polecam najbardziej**. Wygląda i działa dokładnie tak, jak dzisiejsza praca w WSL: to samo okno, te same pliki, ten sam terminal. Różnica jest jedna: „komputer” pod spodem stoi w serwerowni Google i ma na przykład 16 GB pamięci zamiast 3,5 GB. To znaczy koniec z kompilacją na jednym rdzeniu i koniec z zawieszaniem się przy większych zadaniach.
4. **Środowisko w przeglądarce** (Cloud Shell) — istnieje, ale do tego projektu niepotrzebne.

Jedna praktyczna pułapka, o której warto wiedzieć zawczasu: klucze dostępowe generowane w Linuksie pod Windowsem (WSL) lądują w innym miejscu, niż szuka ich VS Code w wersji dla Windows. Jeśli połączenie nie zadziała za pierwszym razem, to najpewniej dlatego — wystarczy skopiować pliki kluczy. Dokładna instrukcja jest w repozytorium.

Co gdzie postawić

Dwa silniki wymagają dwóch różnych układów — i to nie jest widzimisię, tylko wynika z tego, jak są zbudowane:

Ballista wystarczy postawić na **jednej albo kilku zwykłych maszynach w chmurze**. Ballista ma gotową, opisaną w dokumentacji obsługę kontenerów, więc uruchamia się to jedną komendą.

Sail wymaga Kubernetesa — i to nie jest wybór, tylko konieczność. Jak ustaliliśmy wcześniej (Rozdział 3), Sail ma tylko dwa sposoby uruchamiania robotników: „wszyscy w jednym procesie” albo „każdy jako osobny kontener zarządzany przez Kubernetes”. Trzeciej opcji nie ma. Właśnie dlatego lokalna próba się nie powiodła — Kubernetes nie zmieścił się w 3,5 GB pamięci. W chmurze ten problem znika.

Co to jest Kubernetes (raz jeszcze, krótko)

To „brygadzysta” dla kontenerów. Sam decyduje, na której maszynie co uruchomić, restartuje to, co padło, i dokłada mocy, gdy trzeba. Google udostępnia go jako gotową usługę (GKE), więc nie trzeba go instalować ani utrzymywać — wystarczy poprosić o klaster jedną komendą.

Co jest już przygotowane

W repozytorium leży katalog `deploy/` z gotowymi „przepisami”: jak zapakować oba silniki do kontenerów, jak poprosić Google o maszyny, jak wgrać dane. **Nic z tego nie zostało jeszcze uruchomione w chmurze** — to punkt startu na moment, gdy będzie dostęp do projektu i budżetu.

Ile to kosztuje

Co	Ile mniej więcej
Zwykła maszyna (2 rdzenie, 4 GB)	ok. 0,055 USD za godzinę
Mocniejsza maszyna (4 rdzenie, 16 GB)	ok. 0,15 USD za godzinę
Kubernetes	pierwszy klaster za darmo , płaci się tylko za maszyny
Nowe konto	300 USD kredytu na start (90 dni)

Dla porównania: cały dzień pracy na mocniejszej maszynie to jakieś 1,2 USD. Kredyt na start spokojnie wystarczy na wszystkie eksperymenty tej pracy.

Dwie zasady, które najbardziej chronią budżet: **wyłączaj maszyny, gdy ich nie używasz** (płaci się za czas działania, nie za samo posiadanie) i **do pomiarów używaj maszyn „z odzysku”** (60-70% taniej; mogą zostać zabrane w trakcie, co przy powtarzalnych testach nie przeszkadza). Warto też od razu ustawić alert mailowy o przekroczeniu budżetu.

Rozdział 7 — Co to wszystko znaczy dla całej pracy magisterskiej

Promotor postawił jasne wymagania: mechanizm dodawania własnych funkcji do silnika rozproszonego bez jego modyfikowania (forkowania), i to dla **obu** silników — Ballista i Sail. Oba te warunki zostały spełnione dla wszystkich pięciu podstawowych operacji genomycznych (overlap, merge, nearest, coverage, subtract).

Co więcej, **wszystkie pięć liczy się dziś w Ballistrze rozproszone**, a nie tylko jedna, jak było na wcześniejszym etapie. Cztery z nich (merge, subtract, nearest, coverage) doszły do tego w ostatniej turze prac — i to po tym, jak wcześniejsza ocena uznała to za niewykonalne. Warto z tego wyciągnąć wniosek metodologiczny: ta ocena nie była zmyślona, tylko **przedwczesna** — opierała się na stanie rzeczy sprzed pewnej zmiany (zrobienia lokalnej kopii biblioteki) i nie została ponownie sprawdzona po tej zmianie.

Piąta operacja z biblioteki, cluster, pozostaje niewykonalna — ale z konkretnego, dobrze zrozumianego powodu (jej algorytm zakłada, że wszyscy pracują we wspólnej pamięci), co samo w sobie jest wynikiem wartym opisanie.

Po stronie Saila najważniejsze okazało się coś, czego nikt nie planował: **naprawienie własnej pomyłki**. Objaw, który przez kilka etapów pracy przypisywaliśmy błędowi Saila, okazał się ograniczeniem polars-bio. Po jego obejściu Sail odzyskał równoległość, a praca — uczciwe podstawy do porównania obu silników.

Właściwe „wpisanie” tego mechanizmu na stałe do samej biblioteki polars-bio (żeby użytkownik mógł po prostu napisać `pb.overlap(..., engine="ballista")`) zostało świadomie odłożone — bo to decyzja, którą powinien podjąć promotor jako współautor tej biblioteki, a nie coś, co powinno zostać „przy okazji” wpisane na stałe bez jego zgody.

Do zrobienia zostają przede wszystkim: **prawdziwe pomiary wydajności** na większych danych i większej liczbie komputerów — i dopiero teraz mają one sens po obu stronach, bo oba silniki faktycznie coś zrównoleglają. Najpewniej w chmurze, bo lokalna maszyna, jak pokazały te rozdziały, po prostu nie ma na to zasobów; przepisy wdrożeniowe są już przygotowane (Rozdział 6).

Poza tym: zgłoszenie obu drobnych poprawek widoczności autorom biblioteki (po ich przyjęciu lokalna kopia przestanie być potrzebna) oraz rozmowa z promotorem — zarówno podsumowująca te wyniki, jak i dotycząca tego, czy warto usunąć wykryte ograniczenie współbieżności w samym polars-bio.